BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO MÔN HỌC
THỰC TẬP CHUYÊN SÂU

"Thiết kế hệ thống đo và hiển thị cường độ ánh sáng sử dụng cảm biến ánh sáng BH1750 và FPGA Tang Nano 9K, hiển thị kết quả lên màn hình LCD"

Họ và tên : Lê Chí Hiếu

Mã sinh viên : B23DCDT089

Nhóm lớp học : 12

Giảng viên giảng dạy: ***TS. Chu Văn Bền***

Hà Nội – 2026

MỤC LỤC

MỤC LỤC ..... **ii**

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT ..... **v**

DANH MỤC HÌNH ẢNH ..... **vi**

DANH MỤC BẢNG BIỂU ..... **vii**

LỜI MỞ ĐẦU **1**

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG ĐO VÀ HIỂN THỊ CƯỜNG ĐỘ ÁNH SÁNG **2**

1.1. Tổng quan về công nghệ FPGA với Tang Nano 9K 2
1.1.1. Tổng quan về FPGA 2
1.1.2. Tổng quan về Tang Nano 9K 4

1.2. Tổng quan về các thiết bị trong hệ thống 7
1.2.1. Tổng quan về cảm biến ánh sáng BH1750 7
1.2.2. Tổng quan về màn hình LCD 16x2 7

1.3. Giới thiệu về giao thức I2C 8
1.3.1. Giao thức I2C là gì? 8
1.3.2. Đặc điểm cấu tạo: 8
1.3.3. Frame truyền của I2C 10
1.3.4. Thông số kỹ thuật 11
1.3.5. Ứng dụng 12

1.4. Kết luận chương 1 12

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ PHƯƠNG PHÁP NGHIÊN CỨU **13**

2.1. Sơ đồ khối tổng quan hệ thống 13

2.2. Xây dựng module I2C trên FPGA 14
2.2.1. Module truyền dữ liệu I2C 14
2.2.2. Module nhận dữ liệu I2C 17

2.3. Thiết kế giao thức I2C kết nối cảm biến ánh sáng BH1750 19

2.4. Thiết kế giao thức I2C kết nối màn hình LCD 20
2.4.1. Các tín hiệu trong Waveform 21
2.4.2. Phân tích các giai đoạn trong waveform 21

2.5. Kết luận chương 2 26

CHƯƠNG 3: TÍCH HỢP, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG **27**

3.1. Tích hợp và thử nghiệm 27

3.1.1. Tích hợp hệ thống 27
3.1.2. Thử nghiệm hệ thống và kết quả 28
3.2. Đánh giá hiệu suất hệ thống 32
3.2.1. Độ chính xác 32
3.2.2. Độ ổn định 33
3.2.3. Thời gian phản hồi 33
3.2.4. Khả năng mở rộng và tính ứng dụng 33
3.3. Kết luận chương 3 33
TÀI LIỆU THAM KHẢO 35

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

FPGA	Field-Programmable Gate Array (Mảng cổng lập trình được trường)
Gowin EDA	Gowin Electronic Design Automation
HDL	Hardware Description Language (Ngôn ngữ mô tả phần cứng)
I2C	Inter-Integrated Circuit (Giao tiếp liên vi mạch)
SCL	Serial Clock (Đường xung nhịp nối tiếp - trong I2C)
SDA	Serial Data (Đường dữ liệu nối tiếp - trong I2C)
LCD	Liquid Crystal Display (Màn hình tinh thể lỏng)
BH1750	- (Cảm biến ánh sáng)
LUT	Look-Up Table (Bảng tra)
FF	Flip-Flop (Mạch lật / Phần tử nhớ / Thanh ghi)
I/O	Input/Output (Vào/Ra)
GPIO	General-Purpose Input/Output (Chân Vào/Ra đa dụng)
USB	Universal Serial Bus (Bus nối tiếp đa năng)
UART	Universal Asynchronous Receiver/Transmitter
VCC	Voltage Common Collector (Chân nguồn dương)
GND	Ground (Chân đất / Nối đất)

DANH MỤC HÌNH ẢNH

Hình 1.1: Kiến trúc tổng quan của FPGA 3
Hình 1.2: Khối logic lập trình được của FPGA 3
Hình 1.3: Hình ảnh Tang Nano 9k 4
Hình 1.4: Các khối bo mạch trên Tang Nano 9K 6
Hình 1.5: Cảm biến ánh sáng BH1750 7
Hình 1.6: Sơ đồ chân màn hình LCD 16x2 8
Hình 1.7: Cấu trúc bus I2C 9
Hình 1.8: Kết nối Multi-Master trong giao tiếp I2C 9
Hình 1.9: Định dạng truyền dữ liệu I2C 9
Hình 1.10: Sơ đồ chân I2C 10
Hình 1.11: Master gửi dữ liệu đến Slave 10
Hình 1.12: Master đọc dữ liệu từ Slave 11
Hình 2.1: Sơ đồ khối tổng quan hệ thống 13
Hình 2.2: Sơ đồ trạng thái FSM I2C_TX 14
Hình 2.3: Mô phỏng đường dữ liệu SDA, clock SCL và các tín hiệu nội Master.....16
Hình 2.3: Sơ đồ trạng thái FSM I2C_RX 17
Hình 2.5: Mô phỏng đường dữ liệu SDA, clock SCL và các tín hiệu nội Master 19
Hình 2.4: Mô phỏng tín hiệu kết nối I2C với màn hình LCD trên Quatus 20
Hình 3.1: Sơ đồ kết nối các thiết bị phần cứng trong hệ thống 27
Hình 3.2: Mô hình hệ thống thực tế 27
Hình 3.3: Tài nguyên phần cứng sử dụng trên Tang Nano 9k 28
Hình 3.4: Phần mềm Quartus Prime Lite Edition 29
Hình 3.5: Phần mềm Gowin FPGA Designer 29
Hình 3.6: Phân tích tín hiệu bus I2C trên Logic Analyzer 30
Hình 3.7: Phần mềm đo ánh sáng trên điện thoại Light Meter 30

DANH MỤC BẢNG BIỂU

Bảng 1: Thông số kỹ thuật chi tiết của Tang Nano 9K. 5
Bảng 2: Chức năng của các chân I2C 10
Bảng 3: Bảng lệnh cảm biến ánh sáng BH1750 19
Bảng 4: Bảng lệnh điều khiển màn hình LCD 16x2 25
Bảng 5: Bảng đo các chế độ phân giải của cảm biến ánh sáng BH1750 31
Bảng 6: Bảng đánh giá tốc độ lấy mẫu của hệ thống 31
Bảng 7: Bảng đo cường độ ánh sáng 32

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ tự động hóa và các hệ thống thông minh ngày càng thâm nhập sâu rộng vào đời sống, việc giám sát chính xác các thông số môi trường theo thời gian thực đã trở thành một yêu cầu cơ bản và quan trọng. Trong đó, cường độ ánh sáng là một yếu tố vật lý then chốt, ảnh hưởng trực tiếp đến nhiều lĩnh vực đa dạng như nông nghiệp công nghệ cao, quản lý năng lượng trong tòa nhà thông minh, tối ưu hóa quy trình công nghiệp và các nghiên cứu khoa học.

Cảm biến ánh sáng được lựa chọn là BH1750, một linh kiện hiện đại cung cấp giá trị đo cường độ ánh sáng (đơn vị lux) trực tiếp thông qua giao tiếp kỹ thuật số I2C. Việc này không chỉ đảm bảo độ chính xác cao mà còn đơn giản hóa đáng kể quá trình thu thập và xử lý dữ liệu so với các cảm biến analog truyền thống.

Đề tài: "Thiết kế hệ thống đo và hiển thị cường độ ánh sáng sử dụng cảm biến ánh sáng BH1750 và FPGA Tang Nano 9K, hiển thị kết quả lên màn hình LCD" không chỉ mang ý nghĩa là một bài tập ứng dụng kỹ thuật mà còn góp phần đáp ứng nhu cầu thực tiễn về một thiết bị đo ánh sáng nhỏ gọn, chính xác và tiện dụng. Thông qua quá trình thực hiện, chúng tôi mong muốn chứng minh tính khả thi và hiệu quả của việc tích hợp cảm biến BH1750 và màn hình LCD với nền tảng FPGA chi phí hợp lý như Tang Nano 9K, đồng thời tạo ra một sản phẩm mẫu hữu ích có thể phục vụ cho mục đích học tập, nghiên cứu hoặc làm cơ sở phát triển các ứng dụng liên quan trong tương lai. Báo cáo sẽ được trình bày trong 3 chương chính:

- Chương 1: Tổng quan về hệ thống đo và hiển thị cường độ ánh sáng.
- Chương 2: Thiết kế hệ thống và phương pháp nghiên cứu.
- Chương 3: Tích hợp, thử nghiệm và đánh giá hệ thống.

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG ĐO VÀ HIỂN THỊ CƯỜNG ĐỘ ÁNH SÁNG

1.1. Tổng quan về công nghệ FPGA với Tang Nano 9K

1.1.1. Tổng quan về FPGA

Lịch sử ra đời của FPGA:

FPGA được thiết kế đầu tiên bởi Ross Freeman, người sáng lập công ty Xilinx vào năm 1984, kiến trúc mới của FPGA cho phép tích hợp số lượng tương đối lớn các phần tử bán dẫn vào 1 vi mạch so với kiến trúc trước đó là CPLD. FPGA có khả năng chứa tối từ 100.000 đến hàng vài tỷ cổng logic, trong khi CPLD chỉ chứa từ 10.000 đến 100.000 cổng logic; con số này đối với PAL, PLA còn thấp hơn nữa chỉ đạt vài nghìn đến 10.000.

CPLD được cấu trúc từ số lượng nhất định các khối SPLD (*Simple Programable Logic Device*) thuật ngữ chung chỉ PAL, PLA. SPLD thường là một mảng logic AND/OR lập trình được có kích thước xác định và chứa một số lượng hạn chế các phần tử nhớ đồng bộ (*clocked register*). Cấu trúc này hạn chế khả năng thực hiện những hàm phức tạp và thông thường hiệu suất làm việc của vi mạch phụ thuộc vào cấu trúc cụ thể của vi mạch hơn là vào yêu cầu bài toán.

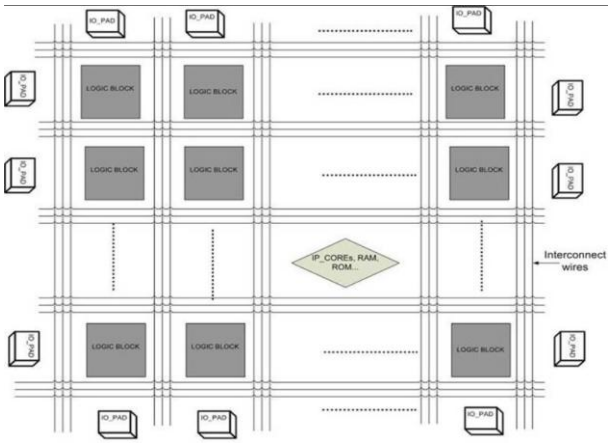
Kiến trúc của FPGA là kiến trúc mảng các khối logic, mỗi khối này nhỏ hơn nhiều nếu đem so sánh với một khối SPLD, ưu điểm này giúp FPGA có thể chứa nhiều hơn các phần tử logic và phát huy tối đa khả năng lập trình của các phần tử logic và hệ thống mạch kết nối, dễ đạt được hiệu năng tối ưu. Kiến trúc của FPGA phức tạp hơn nhiều so với CPLD.

Một điểm khác biệt nữa với CPLD là trong những FPGA hiện đại được tích hợp nhiều bộ nhớ hơn so với đã được tích hợp, họ trở RAM, ROM, tốc độ cao, hay các bộ nhớ chuyên dụng cho những ứng dụng đặc biệt.

Ngoài khả năng cấu trúc lại vi mạch ở mức toàn cục, một số FPGA hiện đại còn hỗ trợ cấu trúc lại ở mức cục bộ, tức là khả năng cấu trúc lại một bộ phận riêng lẻ trong khi vẫn đảm bảo hoạt động bình thường cho các bộ phận khác.

Khái niệm cơ bản và cấu trúc của FPGA:

FPGA (*Field-Programmable Gate Array*) là một vi mạch bán dẫn có thể được lập trình sau khi sản xuất, nghĩa là người dùng có thể tùy chỉnh cấu trúc mạch logic theo nhu cầu sử dụng cụ thể mà không phụ thuộc vào thiết kế ban đầu của nhà sản xuất. Tên gọi "Field-Programmable" nghĩa là thiết bị này có thể lập trình được sau khi đã được sản xuất (thường ngay tại hiện trường hoặc trong hệ thống). Kiến trúc tổng quan của FPGA được mô tả như hình 1.1 :

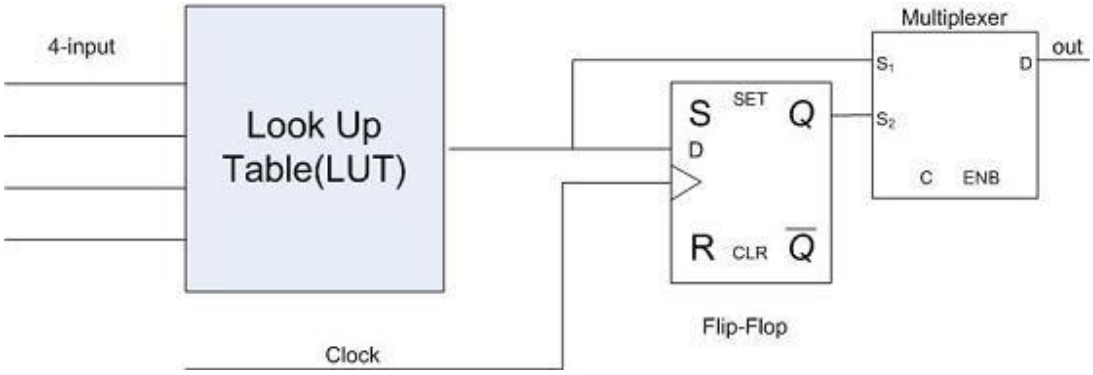


Hình 1.1: Kiến trúc tổng quan của FPGA

Vi mạch FPGA được cấu thành từ các bộ phận:

Các khối logic cơ bản lập trình được (logic block): Khối logic được cấu thành từ LUT và một phần tử nhớ đồng bộ flip-flop. LUT (Look up table) là khối logic có thể thực hiện bất kì hàm logic nào từ 4 đầu vào, kết quả của hàm này tùy vào mục đích mà gửi ra ngoài khối logic trực tiếp hay thông qua phần tử nhớ flip-flop.

Khối logic được mô tả như hình 1.2:



Hình 1.2: Khối logic lập trình được của FPGA

Trong tài liệu hướng dẫn của các dòng FPGA của Xilinx còn sử dụng khái niệm SLICE, 1 Slice gồm 4 khối logic tạo thành, số lượng các Slices thay đổi từ vài nghìn đến vài chục nghìn tùy theo loại FPGA.

Hệ thống mạch liên kết lập trình được: Mạng liên kết trong FPGA được cấu thành từ các đường kết nối theo hai phương ngang và đứng, tùy theo từng loại FPGA mà các đường kết nối được chia thành các nhóm khác nhau, ví dụ trong XC4000 của Xilinx có 3 loại kết nối: ngắn, dài và rất dài. Các đường kết nối được nối với nhau thông qua các khối chuyển mạch lập trình được (programable switch); trong một khối chuyển mạch chứa một số lượng nút chuyển lập trình được, đảm bảo cho các dạng liên kết phức tạp khác nhau."

Khối vào/ra (*IOPads*):

Khối vào/ra nhiều hay ít là tùy thuộc vào từng loại FPGA. Chúng có thể được kết nối với các thiết bị bên ngoài như LED, USB, RS232, RAM...tùy theo mục đích sử dụng.

Các phần tử tích hợp sẵn:

Ngoài các khối logic, tùy theo các loại FPGA khác nhau mà có các phần tử tích hợp thêm khác nhau , ví dụ để thiết kế những ứng dụng SoC , trong dòng Virtex 4, 5 của "Xilinx" có chứa nhân xử lý PowerPC, hay cho những ứng dụng xử lý tín hiệu số trong FPGA được tích hợp các DSP Slice là bộ nhân, cộng tốc độ cao, thực hiện hàm A*B+C, ví dụ dòng Virtex của Xilinx chứa từ vài chục đến hàng trăm DSP slices với A, B, C 18-bit.

Các ứng dụng của FPGA:

Ứng dụng của FPGA bao gồm: xử lý tín hiệu số, các hệ thống hàng không, vũ trụ, quốc phòng, tiến thiết kế mẫu ASIC (*ASIC prototyping*), các hệ thống điều khiển trực quan, phân tích nhận dạng ảnh, nhận dạng tiếng nói, mật mã học, mô hình phần cứng máy tính...

Do tính linh động cao trong quá trình thiết kế cho phép FPGA giải quyết lớp những bài toán phức tạp mà trước kia chỉ thực hiện nhờ phần mềm máy tính, ngoài ra nhờ mật độ cổng logic lớn FPGA được ứng dụng cho những bài toán đòi hỏi khối lượng tính toán lớn và dùng trong các hệ thống làm việc theo thời gian thực.

1.1.2 Tổng quan về Tang Nano 9K

Giới thiệu:

Hình 1.3: Hình ảnh Tang Nano 9k

Tang Nano 9K là một bo mạch phát triển FPGA nhỏ gọn, hiệu suất cao, được trang bị chip FPGA GW1NR-9 từ Gowin Semiconductor. Với thiết kế đa năng, bo mạch này tích hợp nhiều tính năng nổi bật, phù hợp cho cả ứng dụng học tập, nghiên cứu lẫn phát triển các dự án thực tiễn. Tang Nano 9K sở hữu 8640 đơn vị logic LUT4, xung nhịp 27 MHz trên bo mạch cùng 2 PLL, cung cấp khả năng xử lý linh hoạt cho các thiết kế FPGA cơ bản và các ứng dụng phức tạp hơn như triển khai softcore RISC-V (ví dụ: PicoRV).

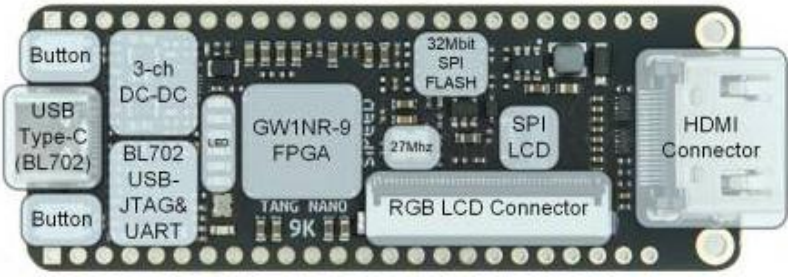
Bo mạch được trang bị các giao diện kết nối phổ biến, bao gồm HDMI, màn hình RGB, và giao diện màn hình SPI, cho phép xuất tín hiệu hình ảnh chất lượng cao hoặc kết nối với các thiết bị hiển thị đa dạng. Ngoài ra, Tang Nano 9K có bộ nhớ flash SPI 32 Mbit để lưu trữ cấu hình FPGA, cùng 6 đèn LED lập trình được và 2 nút nhấn để tương tác người dùng. Hệ thống I/O phong phú hỗ trợ các mức dòng 4mA, 8mA, 16mA, 24mA, với các tùy chọn linh hoạt như Bus Keeper, điện trở kéo lên/kéo xuống, và Open Drain, đáp ứng nhiều nhu cầu kết nối ngoại vi.

Thông số kỹ thuật:

Bảng 1: Thông số kỹ thuật chi tiết của Tang Nano 9K.

Mục	Giá trị
Logic units (LUT4)	8640
Registers (FF)	6480
ShadowSRAM SSRAM (bits)	17280
Block SRAM BSRAM (bits)	468K
Number of B-SRAM	26
User flash(bits)	608K
SDR SDRAM (bits)	64M
18 x 18 Multiplier	20
SPI FLASH	32M-bit
Number of PLL	2
Display interface	Giao diện HDMI, giao diện màn hình SPI và giao diện màn hình RGB
Debugger	Chip BL702 tích hợp cung cấp chức năng USB-JTAG và USB-UART cho GW1NR-9
IO	• hỗ trợ 4mA、8mA、16mA、24mA các khả năng điều khiển khác • Cung cấp các tùy chọn đầu ra Bus Keeper, điện trở kéo lên/kéo xuống và Open Drain độc lập cho mỗi I/O
Connector	Khe cắm thẻ TF, 2x24P 2.54mm Miếng đệm đầu cắm
Button	2 nút có thể lập trình cho người dùng
LED	Có 6 đèn LED có thể lập trình trên bo mạch

Các khối chức năng trên bo mạch:



Hình 1.4: Các khối bo mạch trên Tang Nano 9K

Trên bo mạch này có các thành phần chính đáng chú ý:

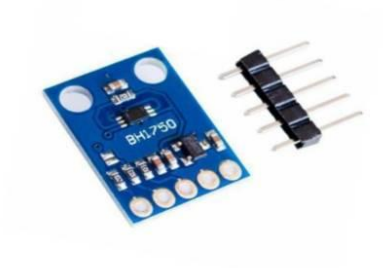
- GW1NR-9 FPGA: Đây là chip FPGA chính, nơi chứa logic xử lý của hệ thống đo ánh sáng.
- USB Type-C (BL702): Cổng này dùng để cấp nguồn cho bo mạch và quan trọng nhất là để nạp chương trình (bitstream) từ máy tính vào FPGA thông qua chip BL702 tích hợp (đóng vai trò giao tiếp USB-JTAG/UART). Điều này tương ứng với khối "Máy tính" và chức năng "Nạp chương trình & Nguồn" trong sơ đồ của bạn.
- 32Mbit SPI FLASH: Bộ nhớ flash để lưu trữ file cấu hình FPGA, giúp bo mạch tự động chạy chương trình của bạn sau mỗi lần cấp nguồn mà không cần nạp lại từ máy tính.
- HDMI Connector: Cổng xuất tín hiệu hình ảnh kỹ thuật số. Bo mạch này có khả năng xuất ra màn hình HDMI.
- RGB LCD Connector: Cổng kết nối chuyên dụng cho các màn hình LCD giao tiếp song song (RGB interface).
- SPI LCD: Chỉ dẫn cho thấy có hỗ trợ kết nối màn hình LCD qua giao tiếp SPI (thường dùng các chân GPIO).
- Buttons: Các nút nhấn để người dùng có thể tương tác (ví dụ: reset, hoặc làm đầu vào cho chương trình).
- 27Mhz Oscillator: Bộ tạo dao động cung cấp xung nhịp cơ bản cho FPGA hoạt động.
- Hàng chân GPIO (không ghi chú rõ nhưng là các chân dọc 2 bên): Đây là nơi bạn sẽ kết nối các thành phần ngoại vi như Cảm biến ánh sáng BH1750 và Màn hình LCD hiển thị kết quả (loại dùng giao tiếp I2C như trong sơ đồ) vào bo mạch FPGA. FPGA sẽ điều khiển các chân GPIO này để thực hiện giao tiếp I2C (SDA, SCL).
- Và một số các linh kiện khác trên bo mạch.

1.2. Tổng quan về các thiết bị trong hệ thống

1.2.1. Tổng quan về cảm biến ánh sáng BH1750

Mô tả cảm biến:

- Cảm biến cường độ sáng BH1750 là một cảm biến kỹ thuật số được sử dụng để đo lường cường độ sáng trong môi trường. Điểm đánh giá của BH1750 bao gồm:
- Độ Chính Xác: BH1750 cung cấp kết quả đo chính xác với độ phân giải cao, cho phép đo lường cường độ sáng trong mức độ rộng từ môi trường ánh sáng yếu đến môi trường ánh sáng mạnh.
- Giao Tiếp I2C: BH1750 sử dụng giao tiếp truyền thống I2C, cho phép kết nối dễ dàng với các vi điều khiển như Arduino.
- Tiết Kiệm Năng Lượng: Cảm biến được thiết kế để tiêu thụ ít năng lượng, điều này phù hợp cho các ứng dụng yêu cầu tiết kiệm năng lượng và ứng dụng di động.
- Khả Năng Điều Chỉnh: BH1750 có thể được điều chỉnh để cấu hình độ nhạy và thời gian tích hợp, tùy thuộc vào nhu cầu cụ thể của ứng dụng.



Hình 1.5: Cảm biến ánh sáng BH1750

Thông số kỹ thuật:

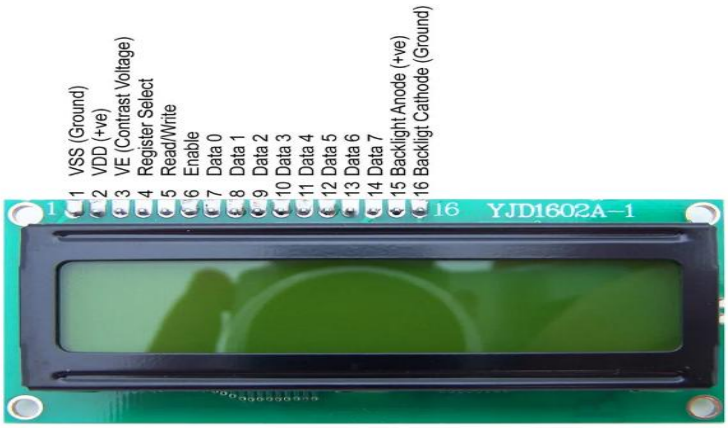
- Nguồn: 3 -> 5VDC
- Giao tiếp: I2C
- Khoảng đo: 1 -> 65535 lux
- Kích cỡ: 21x6x3.3 mm

1.2.2. Tổng quan về màn hình LCD 16x2

Thông số kỹ thuật của màn hình LCD 16x2:

- Điện áp hoạt động: 2.5-6V DC.
- Hỗ trợ màn hình: LCD1602,1604,2004 (driver HD44780).
- Giao tiếp: I2C.
- Địa chỉ mặc định: 0X27 (có thể điều chỉnh bằng ngắn mạch chân A0/A1/A2).
- Tích hợp Jump chốt để cung cấp đèn cho LCD hoặc ngắt.
- Tích hợp biến trở xoay điều chỉnh độ tương phản cho LCD.

Sơ đồ chân của màn hình LCD 16x2:



Hình 1.6: Sơ đồ chân màn hình LCD 16x2

Các tính năng của màn hình LCD 16x2:

- Nó có thể hiển thị 32 ký tự, mỗi ký tự gồm 5x8 điểm ảnh.
- Nó có thể hoạt động ở hai chế độ: 8-bit và 4-bit, tùy thuộc vào số đường dữ liệu được sử dụng.
- Nó có một bộ điều khiển HD44780 bên trong, có thể nhận và xử lý các lệnh và dữ liệu từ vi điều khiển.
- Nó có thể hiển thị các ký tự và biểu tượng đơn giản, phù hợp cho các ứng dụng như viễn thông, máy tính, điện tử tiêu dùng và IoT.

Ứng dụng của màn hình LCD 16x2:

- Hiển thị thông tin trạng thái, giá trị cảm biến, thời gian, ngày tháng, nhiệt độ, độ ẩm, áp suất, tốc độ, khoảng cách, v.v.
- Tạo giao diện người dùng, menu, nút nhấn, bàn phím, chuông báo, v.v.
- Tạo các dự án sáng tạo, vui nhộn, giáo dục, như đồng hồ đếm ngược, đồng hồ báo thức, máy tính, máy trò chơi, ...

1.3. Giới thiệu về giao thức I2C

1.3.1. Giao thức I2C là gì?

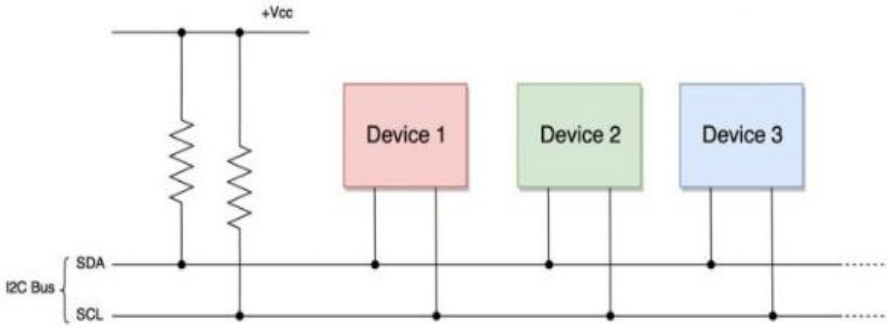
Giao thức I2C (Inter-Integrated Circuit) là một giao thức truyền thông nối tiếp được thiết kế để cho phép các vi mạch (IC) giao tiếp với nhau trong các hệ thống nhúng, sử dụng hai dây dẫn để truyền dữ liệu và đồng bộ hóa.

1.3.2. Đặc điểm cấu tạo:

Cấu trúc bus:

- SDA (Serial Data Line): Đường truyền dữ liệu hai chiều, mang thông tin giữa master và slave.
- SCL (Serial Clock Line): Đường truyền tín hiệu xung nhịp, được tạo bởi master để đồng bộ hóa quá trình truyền dữ liệu.
- Pull-up resistors: Cả SDA và SCL đều được kéo lên mức logic cao (thường là 3.3V hoặc 5V) thông qua các điện trở kéo lên (pull-up resistors). Điều này đảm bảo bus ở trạng thái "cao" khi không có thiết bị nào kéo xuống mức "thấp".

- Open-drain/Open-collector: Các thiết bị trên bus sử dụng cấu hình open-drain (hoặc open-collector), nghĩa là chúng chỉ có thể kéo tín hiệu xuống mức thấp (0), còn mức cao (1) được cung cấp bởi điện trở kéo lên. Như hình 1.7 sau:



Hình 1.7: Cấu trúc bus I2C

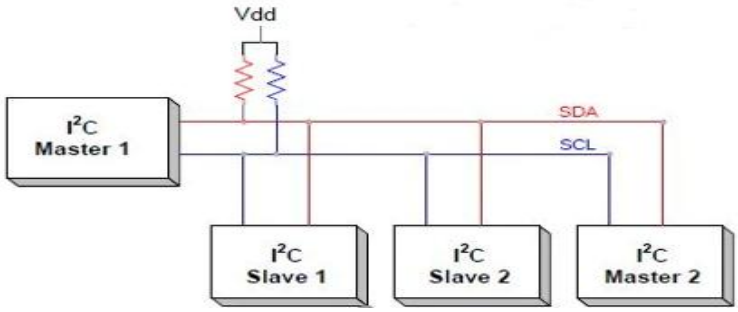
Master:

- Khởi tạo giao tiếp bằng cách tạo tín hiệu Start và Stop.
- Cung cấp tín hiệu SCL để đồng bộ hóa.
- Gửi địa chỉ slave và dữ liệu.

Slave:

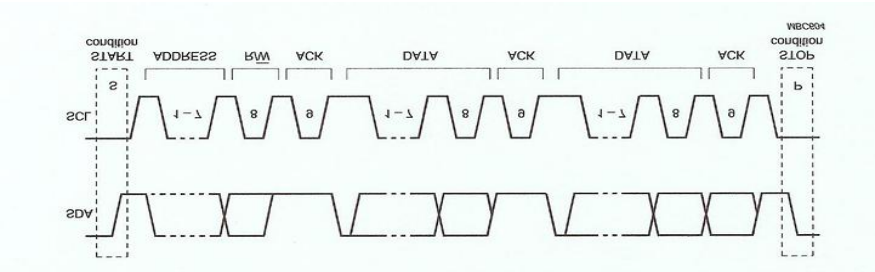
- Lắng nghe bus và phản hồi khi địa chỉ của nó được gọi.
- Nhận hoặc truyền dữ liệu theo lệnh từ master.

Truyền dữ liệu được mô tả như hình 1.8 sau:



Hình 1.8: Kết nối Multi-Master trong giao tiếp I2C

Truyền dữ liệu được mô tả như hình sau:



Hình 1.9: Định dạng truyền dữ liệu I2C

Dữ liệu được truyền theo từng bit, đồng bộ với xung nhịp SCL:

- Khi SCL ở mức cao, dữ liệu trên SDA phải ổn định (không thay đổi).

- Khi SCL chuyển từ cao xuống thấp, dữ liệu trên SDA có thể thay đổi để chuẩn bị cho bit tiếp theo.
- Mỗi byte dữ liệu (8 bit) được gửi kèm theo một bit ACK/NACK (xác nhận/không xác nhận) từ phía nhận.

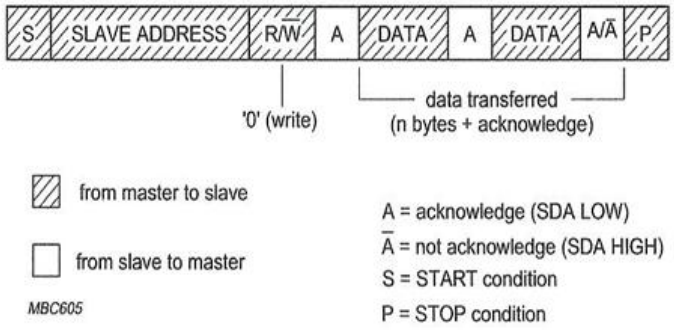
Sơ đồ chân:

Hình 1.10: Sơ đồ chân I2C

Bảng 2: Chức năng của các chân I2C

Tên chân	Chức năng
16 chân	Kết nối với 16 chân của LCD
VCC	Chân nguồn
GND	Chân đất
SDA	Đường truyền nhận dữ liệu.
SCL	Tạo xung nhịp đồng hồ do Master phát đi

1.3.3. Frame truyền của I2C

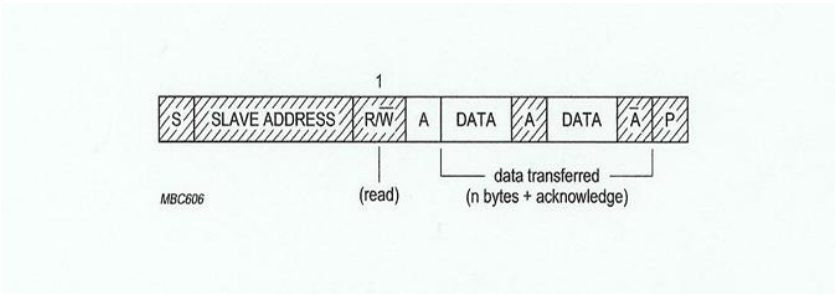


Hình 1.11: Master gửi dữ liệu đến Slave

Quá trình Master gửi dữ liệu tới Slave:

- Tín hiệu Start (S): Master khởi tạo giao tiếp bằng cách gửi tín hiệu Start. Đường SDA chuyển từ mức cao (1) xuống mức thấp (0) trong khi SCL đang ở mức cao (1).
- Gửi địa chỉ Slave: Master gửi địa chỉ 7-bit của Slave mà nó muốn giao tiếp.
- Bit R/W: Bit thứ 8 xác định hướng truyền, R/W = 0 (ghi - "W" - write), nghĩa là Master sẽ gửi dữ liệu đến Slave.

- Slave gửi ACK (A): Sau khi nhận địa chỉ, Slave kiểm tra xem địa chỉ có khớp với địa chỉ của nó hay không. Nếu khớp, Slave kéo SDA xuống mức thấp (0) trong chu kỳ SCL tiếp theo để gửi tín hiệu ACK (xác nhận).
- Master gửi byte dữ liệu (DATA): Master bắt đầu gửi dữ liệu đến Slave, mỗi byte dữ liệu là 8 Bit, Master có thể gửi 1 hoặc nhiều byte liên tiếp.
- Slave gửi ACK sau mỗi byte dữ liệu: A (Acknowledge): Sau mỗi byte dữ liệu, Slave gửi tín hiệu ACK để xác nhận đã nhận thành công. Slave kéo SDA xuống mức thấp (0) trong chu kỳ SCL tiếp theo.
- Tín hiệu Stop (P): Sau khi truyền xong tất cả các byte dữ liệu, Master gửi tín hiệu Stop để kết thúc giao tiếp. Đường SDA chuyển từ mức thấp (0) lên mức cao (1) trong khi SCL đang ở mức cao (1).



Hình 1.12: Master đọc dữ liệu từ Slave

Quá trình Master đọc dữ liệu từ Slave:

- Tín hiệu Start (S): Master khởi tạo giao tiếp bằng cách gửi tín hiệu Start. Đường SDA chuyển từ mức cao (1) xuống mức thấp (0) trong khi SCL đang ở mức cao (1).
- Gửi địa chỉ Slave: Master gửi địa chỉ 7-bit của Slave mà nó muốn giao tiếp.
- Bit R/W: Bit thứ 8 xác định hướng truyền, R/W = 1 (đọc - "R" - read), nghĩa là Master muốn đọc dữ liệu từ Slave.
- Slave gửi ACK (A): Sau khi nhận địa chỉ, Slave kiểm tra xem địa chỉ có khớp với địa chỉ của nó hay không. Nếu khớp, Slave kéo SDA xuống mức thấp (0) trong chu kỳ SCL tiếp theo để gửi tín hiệu ACK (xác nhận).
- Slave gửi byte dữ liệu (DATA): Slave bắt đầu gửi dữ liệu đến Master, mỗi byte dữ liệu là 8 Bit, Slave có thể gửi 1 hoặc nhiều byte liên tiếp.
- Master gửi ACK (A1) sau mỗi byte dữ liệu: Sau mỗi byte dữ liệu (trừ byte cuối), Master gửi tín hiệu ACK để xác nhận đã nhận thành công. Master kéo SDA xuống mức thấp (0) trong chu kỳ SCL tiếp theo.
- Master gửi NACK ($\bar{A}$) cho byte cuối: Khi Master không muốn nhận thêm dữ liệu, nó gửi tín hiệu NACK cho byte cuối cùng, Master giữ SDA ở mức cao (1) trong chu kỳ SCL tiếp theo.
- Tín hiệu Stop (P): Sau khi nhận đủ dữ liệu, Master gửi tín hiệu Stop để kết thúc giao tiếp. Đường SDA chuyển từ mức thấp (0) lên mức cao (1) trong khi SCL đang ở mức cao (1).

1.3.4. Thông số kỹ thuật

Danh sách tốc độ truyền được hỗ trợ:

- Điện áp hoạt động: Thường là 3.3V hoặc 5V, nhưng có thể hỗ trợ các mức điện áp khác tùy thiết bị.
- Tốc độ truyền: I2C hỗ trợ nhiều chế độ tốc độ khác nhau như:
- Standard Mode: 100 kbps.
- Fast Mode: 400 kbps.
- Fast Mode Plus: 1 Mbps.
- High-Speed Mode: Lên đến 3.4 Mbps.

Data Frame:

- Start condition (S): SDA kéo xuống trong khi SCL đang cao.
- Địa chỉ thiết bị (7-bit hoặc 10-bit) + bit đọc/ghi (R/W).
- ACK/NACK: Mỗi byte truyền sẽ nhận được 1 bit ACK hoặc NACK từ phía nhận.
- Dữ liệu: 8-bit dữ liệu mỗi lần truyền.
- Stop condition (P): SDA lên cao trong khi SCL đang cao.

Tính năng đặc biệt:

- Cho phép nhiều thiết bị (multi-slave).
- Hỗ trợ nhiều master (multi-master arbitration).
- Đồng bộ hóa dữ liệu (clock stretching).
- Dễ mở rộng trong mạch nhúng nhỏ gọn.

1.3.5. Ứng dụng

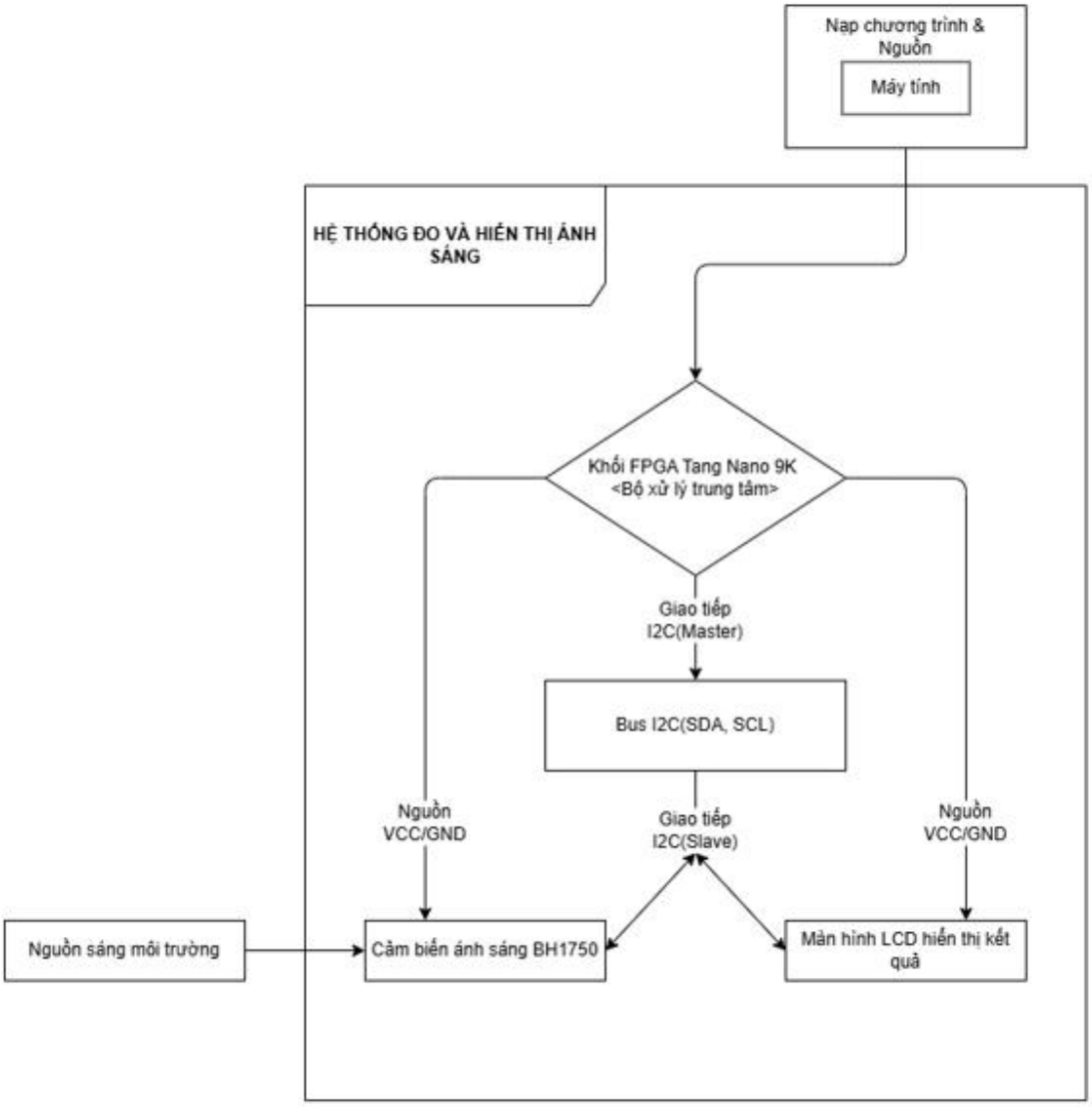
- Điều khiển các chip ngoại vi trong hệ thống như cảm biến, bộ nhớ và các module hiển thị LCD.
- Giao tiếp giữa các vi mạch và linh kiện khác nhau trong hệ thống điện tử, giúp chúng truyền thông tin với nhau.
- Điều khiển các thiết bị ngoại vi thông qua giao thức I2C như các module PWM, ADC, DAC hoặc các bộ chuyển đổi giao diện.

1.4. Kết luận chương 1

Chương 1 đã cung cấp cái nhìn tổng quan về công nghệ FPGA, cụ thể là bo mạch Tang Nano 9K, cùng với các thành phần chính của hệ thống bao gồm cảm biến ánh sáng BH1750 và màn hình LCD 16x2, tất cả được kết nối thông qua giao thức I2C. FPGA được giới thiệu như một nền tảng linh hoạt với khả năng lập trình logic tùy chỉnh, phù hợp cho các ứng dụng thời gian thực như đo lường ánh sáng. Cảm biến BH1750 nổi bật với độ chính xác cao, giao tiếp I2C đơn giản và tiết kiệm năng lượng, trong khi LCD 16x2 đáp ứng tốt nhu cầu hiển thị dữ liệu cơ bản. Giao thức I2C được mô tả chi tiết, nhấn mạnh vai trò của nó trong việc đảm bảo truyền thông hiệu quả giữa các thiết bị.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ PHƯƠNG PHÁP NGHIÊN CỨU

2.1. Sơ đồ khối tổng quan hệ thống



Hình 2.1: Sơ đồ khối tổng quan hệ thống

Hệ thống này hoạt động với FPGA Tang Nano 9K làm bộ xử lý trung tâm, điều khiển việc đọc dữ liệu từ cảm biến ánh sáng BH1750 thông qua giao tiếp I2C (FPGA là Master, cảm biến là Slave). Cảm biến này đo cường độ ánh sáng từ môi trường xung quanh. Dữ liệu sau khi được FPGA xử lý sẽ được gửi đến màn hình LCD (hoạt động như một thiết bị Slave khác trên cùng bus I2C) để hiển thị kết quả đo. Máy tính

được sử dụng để nạp chương trình và cấp nguồn ban đầu cho FPGA, trong khi cảm biến và LCD yêu cầu nguồn VCC/GND riêng.

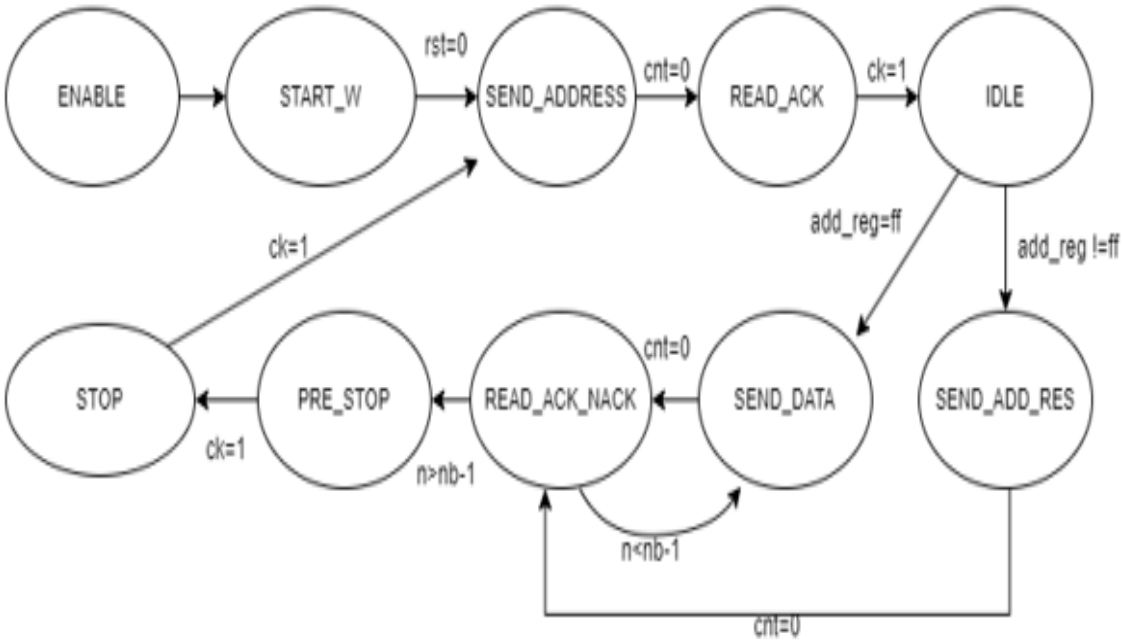
2.2. Xây dựng module I2C trên FPGA

2.2.1. Module truyền dữ liệu I2C

Module truyền dữ liệu I2C(I2C_TX) truyền dữ liệu từ FPGA đến thiết bị slave (như BH1750). FSM điều khiển quá trình truyền bao gồm các trạng thái sau (được định nghĩa trong parameter):

- START_W = -1: Trạng thái khởi tạo (start condition).
- SEND_ADDRESS = 0: Gửi địa chỉ slave (7 bit + bit R/W).
- READ_ACK = 1: Đọc bit ACK từ slave sau khi gửi địa chỉ.
- IDLE = 2: Trạng thái chờ (pause) trước khi gửi dữ liệu hoặc địa chỉ thanh ghi.
- SEND_DATA = 3: Gửi dữ liệu (data byte).
- SEND_ADD_RES = 7: Gửi địa chỉ thanh ghi (register address).
- READ_ACK_NACK = 4: Đọc bit ACK/NACK sau khi gửi dữ liệu hoặc địa chỉ thanh ghi.
- PRE_STOP = 5: Chuẩn bị cho stop condition.
- STOP = 6: Tạo stop condition và kết thúc truyền.

Sơ đồ trạng thái FSM:



Hình 2.2: Sơ đồ trạng thái FSM I2C_TX

- **[0: START_W (-1)]:**
  + Đây là trạng thái khởi tạo ban đầu
  + Hoạt động: Reset các biến (ck = 3, pause = 0, count = 7, state = 0, done = 0, num_byte_cnt = 0).
  + Nếu rst == 0 hoặc enable == 0 -> Chuyển sang SEND_ADDRESS (0).
- **[1: SEND_ADDRESS (0)]:**
  + Gửi địa chỉ slave (7 bit + bit R/W, từ add[7:0]).
  + Hoạt động:
    - Gán temp = add[count] để gửi từng bit qua SDA.
    - Giảm count từ 7 xuống 0.
    - Nếu khi count == 0 và ck == 1 -> Chuyển sang READ_ACK (1).
- **[2: READ_ACK (1)]:**
  + Đọc bit ACK từ slave.

  + Hoạt động:
    - Đặt temp = 1'bz để SDA thành high-impedance, cho phép slave trả về ACK.
    - Lưu giá trị ACK vào checkACK khi ck == 3.
    - Luôn chuyển sang IDLE (2) khi ck == 1.
- **[3: IDLE (2)]:**
  + Trạng thái chờ để đồng bộ trước khi gửi dữ liệu hoặc địa chỉ thanh ghi.

  + Hoạt động:
    - Đặt pause = 4 để tạm dừng 4 chu kỳ.
    - Đặt scl = 0, temp = 1.
    - Khi ck == 1 và pause == 0:
    - Nếu add_reg == 8'hff -> Chuyển sang SEND_DATA (3).
    - Nếu add reg!= 8'hff -> Chuyển sang SEND_ADD_RES (7).
- **[4: SEND_DATA (3)]:**
  + Gửi byte dữ liệu (từ data).

  + Hoạt động:
    - Gán temp = data[count + (num-1)*8] để gửi từng bit.
    - Giảm count từ 7 xuống 0, giảm num (đếm số byte còn lại).
    - Khi count == 0 và ck == 1 -> Chuyển sang READ_ACK_NACK (4).
- **[5: SEND_ADD_RES (7)]:**
  + Gửi địa chỉ thanh ghi (từ add_reg).

  + Hoạt động:
    - Gán temp = add_reg[count] để gửi từng bit.
    - Giảm count từ 7 xuống 0.

- Khi count == 0 và ck == 1 -> Chuyển sang READ_ACK_NACK (4).

- **[6: READ_ACK_NACK (4)]:**

+ Đọc bit ACK/NACK sau khi gửi dữ liệu hoặc địa chỉ thanh ghi.

+ Hoạt động:

- Đặt temp = 1'bz để đọc phản hồi từ slave.
- Tăng num_byte_cnt để đếm số byte đã gửi.

+ Khi ck == 1:

- Nếu num_byte_cnt < num_byte-1 (hoặc add_reg != 8'hff) -> Chuyển sang SEND_DATA (3).
- Nếu num_byte_cnt >= num_byte-1 -> Chuyển sang PRE_STOP (5).

- **[7: PRE_STOP (5)]:**

+ Chuẩn bị tạo stop condition.

+ Hoạt động:

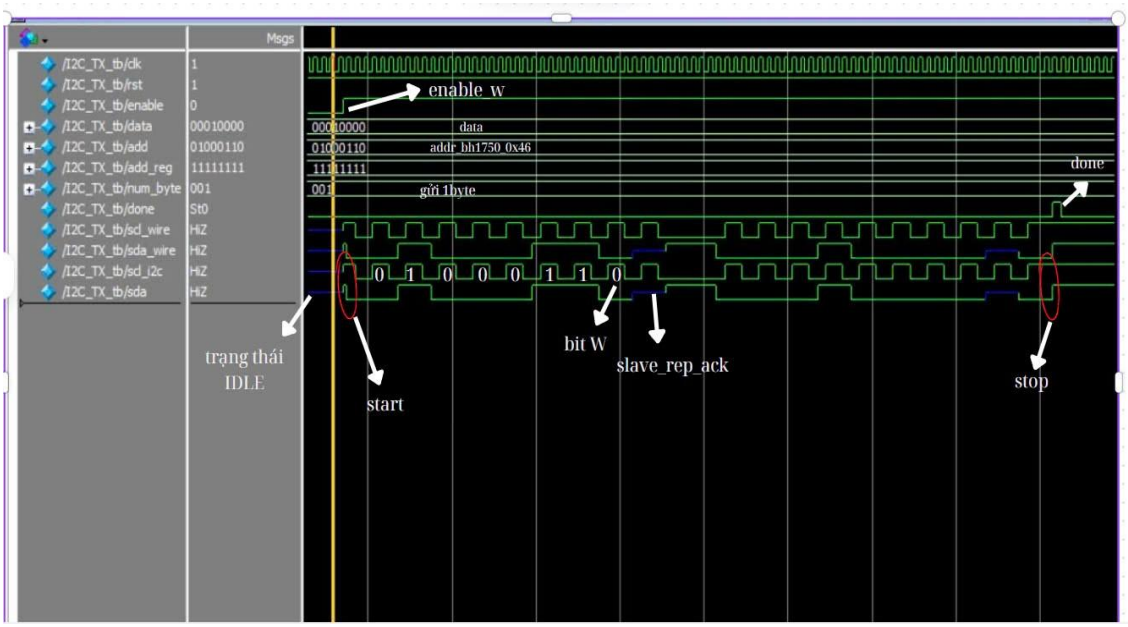
- Đặt temp = 0 (SDA kéo xuống thấp).
- Khi ck == 1 -> Chuyển sang STOP (6).

- **[8: STOP (6)]:**

+ Tạo stop condition và kết thúc truyền.

+ Hoạt động:

- Đặt pause = 60, temp = 1, scl = 1, done = 1.
- Reset num = num_byte_cons.
- Khi ck == 1 -> Chuyển về SEND_ADDRESS (0) cho lần truyền tiếp theo.



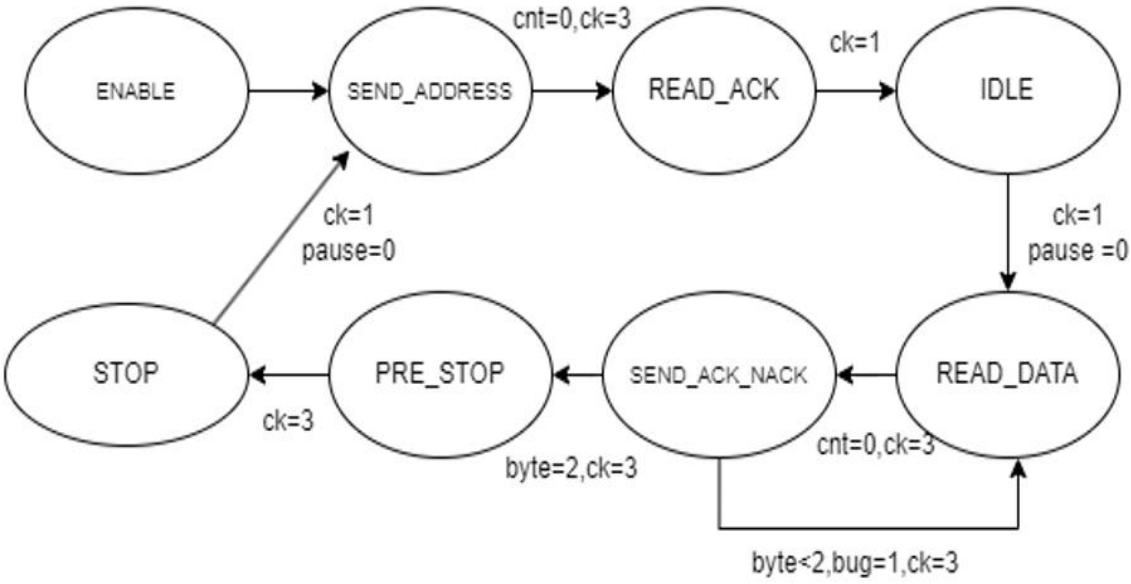
Hình 2.1: Mô phỏng đường dữ liệu SDA, clock SCL và các tín hiệu nội Master truyền

2.2.2. Module nhận dữ liệu I2C

Module nhận dữ liệu I2C(I2C_RX) nhận dữ liệu từ thiết bị slave (như BH1750) đến FPGA. FSM điều khiển quá trình nhận bao gồm các trạng thái sau (được định nghĩa trong parameter):

- START_W = -1: Trạng thái khởi tạo (start condition).
- SEND_ADDRESS = 0: Gửi địa chỉ slave (7 bit + bit R/W).
- READ_ACK = 1: Đọc bit ACK từ slave sau khi gửi địa chỉ.
- IDLE = 2: Trạng thái chờ (pause) trước khi gửi dữ liệu hoặc địa chỉ thanh ghi.
- SEND_DATA = 3: Gửi dữ liệu (data byte).
- SEND_ADD_RES = 7: Gửi địa chỉ thanh ghi (register address).
- READ_ACK_NACK = 4: Đọc bit ACK/NACK sau khi gửi dữ liệu hoặc địa chỉ thanh ghi.
- PRE_STOP = 5: Chuẩn bị cho stop condition.
- STOP = 6: Tạo stop condition và kết thúc truyền.

Sơ đồ trạng thái FSM:



Hình 2.3: Sơ đồ trạng thái FSM I2C_RX

- **SEND_ADDRESS (0)**:

  + Gửi địa chỉ I2C (8 bit từ add[7:0]).

  + Hoạt động:

    - Gửi từng bit của add từ MSB (add[7]) đến LSB (add[0]), với count giảm từ 7 về 0.
    - Khi count == 0 và ck == 3: Chuyển sang READ_ACK (1).
    - Nếu rst == 0 hoặc enable == 0: Quay lại SEND_ADDRESS (0).

- **READ_ACK (1)**:
  + Đọc bit ACK từ slave (qua sda).

  + Hoạt động:

    - Kiểm tra sda để lưu vào checkACK. Trong mã, điều kiện chuyển đổi không phụ thuộc vào checkACK, mà luôn chuyển sang IDLE.
    - Khi ck == 1: Chuyển sang IDLE (2).

  + Lưu ý: Có một nhánh không sử dụng (else begin state <= SEND_ADDRESS) vì điều kiện luôn là 1, nên nhánh này không được kích hoạt.
- **IDLE (2)**:
  + Trạng thái chờ trước khi nhận dữ liệu.

  + Hoạt động:

    - Đặt pause <= 4, scl <= 0, temp <= 1'bz, và ck <= 0.
    - Khi ck == 1 và pause == 0: Chuyển sang SEND_DATA (3)(thực chất là READ_DATA vì module này nhận dữ liệu).
- **SEND_DATA / READ_DATA (3)**:
  + Nhận dữ liệu từ slave (8 bit mỗi lần).

  + Hoạt động:

    - Lưu từng bit từ sda vào data [count + 8 - count_byte*8], với count giảm từ 7 về 0.
    - Khi count == 0 và ck == 3: Chuyển sang READ_ACK_NACK (4).
- **READ_ACK_NACK (4)**:
  + Gửi ACK hoặc NACK đến slave sau khi nhận dữ liệu.

  + Hoạt động:

    - Nếu count_byte == 2: Gửi NACK (temp <= 1'b1) và chuyển sang PRE_STOP.
    - Nếu count_byte < 2: Gửi ACK (temp <= 1'b0) lần đầu, sau đó quay lại SEND_DATA để nhận byte tiếp theo.
    - Khi count_byte == 2 và ck == 3: Chuyển sang PRE_STOP (5).
    - Khi count_byte < 2, checkBug == 1, và ck == 3: Chuyển sang SEND_DATA (3).
    - CheckBug được sử dụng để đảm bảo gửi ACK chỉ một lần trước khi quay lại SEND_DATA.
- **PRE_STOP (5)**:
  + Chuẩn bị cho điều kiện dừng (stop condition).

  + Hoạt động:

    - Đặt temp <= 0 (kéo sda xuống thấp).

- Khi ck == 3: Chuyển sang STOP (6).

- **STOP (6)**:

  + Tạo điều kiện dừng và hoàn thành giao tiếp.

  + Hoạt động:

  - Đặt pause <= 60, temp <= 1'bz, scl <= 1, ck <= 3, và done <= 1.
  - Khi ck == 1 và pause == 0: Chuyển về SEND_ADDRESS (0) để bắt đầu giao tiếp mới.

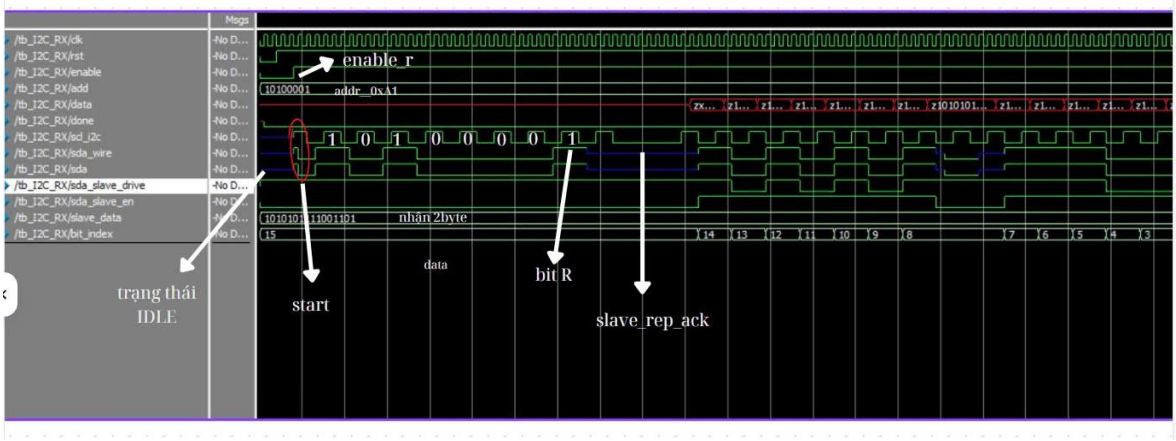
- **Điều kiện bổ sung:**

  + Reset hoặc disable:

  - Nếu rst == 0 hoặc enable == 0 tại bất kỳ trạng thái nào, FSM sẽ quay lại SEND_ADDRESS (0), với các biến được khởi tạo lại (ck <= 3, pause <= 0, count <= 7, count_byte <= 0, done <= 0).

  + Thời gian chờ (pause):

  - Một số trạng thái sử dụng pause để trì hoãn, ví dụ IDLE đặt pause <= 4 và STOP đặt pause <= 60. FSM chỉ chuyển trạng thái khi pause == 0.



Hình 2.2: Mô phỏng đường dữ liệu SDA, clock SCL và các tín hiệu nội Master nhận

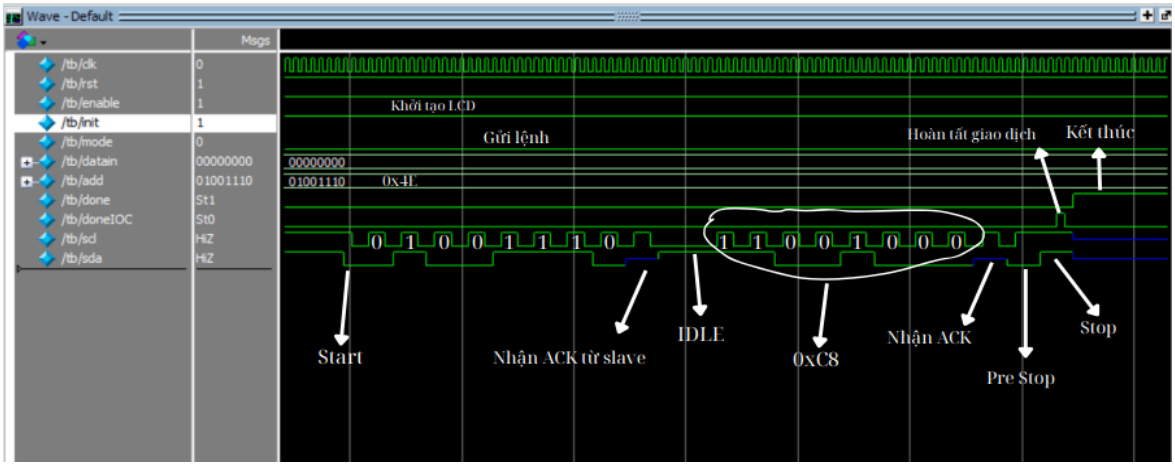
2.3. Thiết kế giao thức I2C kết nối cảm biến ánh sáng BH1750

Bảng 3: Bảng lệnh cảm biến ánh sáng BH1750

Lệnh	Mã Lệnh (Opcode)	Chú Thích
Tắt nguồn (Power Down)	0000_0000	Không ở trạng thái hoạt động.
Bật nguồn (Power On)	0000_0001	Chờ lệnh đo ánh sáng.
Reset	0000_0111	Xóa giá trị thanh ghi dữ liệu. Không dùng được khi đang ở chế độ Power Down.
Chế độ đo liên tục độ phân giải cao	0001_0000	Bắt đầu đo ở độ phân giải 1lx. Thời gian đo khoảng 120ms.

Chế độ đo liên tục độ phân giải cao 2	0001_0001	Bắt đầu đo ở độ phân giải 0.5lx. Thời gian đo khoảng 120ms.
Chế độ đo liên tục độ phân giải thấp	0001_0011	Bắt đầu đo ở độ phân giải 4lx. Thời gian đo khoảng 16ms.
Chế độ đo một lần độ phân giải cao	0010_0000	Đo 1 lần ở độ phân giải 1lx. Thời gian đo khoảng 120ms. Tự chuyển về chế độ Power Down sau khi đo.
Chế độ đo một lần độ phân giải cao 2	0010_0001	Đo 1 lần ở độ phân giải 0.5lx. Thời gian đo khoảng 120ms. Tự chuyển về chế độ Power Down sau khi đo.
Chế độ đo một lần độ phân giải thấp	0010_0011	Đo 1 lần ở độ phân giải 4lx. Thời gian đo khoảng 16ms. Tự chuyển về chế độ Power Down sau khi đo.
Thay đổi thời gian đo (bit cao)	01000_MT [7,6,5]	Thay đổi thời gian đo. *Xem thêm mục điều chỉnh kết quả đo do ảnh hưởng cửa sổ quang học.
Thay đổi thời gian đo (bit thấp)	011_MT[4,3,2,1,0]	Thay đổi thời gian đo. *Xem thêm mục điều chỉnh kết quả đo do ảnh hưởng cửa sổ quang học.

2.4. Thiết kế giao thức I2C kết nối màn hình LCD



Hình 2.4: Mô phỏng tín hiệu kết nối I2C với màn hình LCD trên Quatus

Dự án sử dụng giao thức I2C để giao tiếp giữa FPGA (Tang Nano 9K) và LCD, với cảm biến ánh sáng BH1750. Module I2C_TX được thiết kế để gửi dữ liệu từ master (FPGA) đến slave (LCD), và được tích hợp trong các module LCD_send và LCD_top để khởi tạo và hiển thị dữ liệu trên LCD.

Trong đó:

+ Địa chỉ slave được gửi là 0x4E (7'h27 << 1, bit write = 0).

+ Dữ liệu gửi sau đó là 0xC8 (11001000), được xác định là nibble thấp của lệnh 0x0C trong giai đoạn cuối của quá trình khởi tạo LCD (state 8 của LCD_top).

+ Mode = 0, nghĩa là module đang ở chế độ gửi lệnh (command mode),

Trong module LCD_send, tín hiệu mode quyết định chế độ hoạt động:

+ Mode = 0: Command mode – Gửi lệnh đến LCD (ví dụ: khởi tạo, xóa màn hình, đặt con trỏ).

+ Mode = 1: Data mode – Gửi dữ liệu để hiển thị trên LCD (ví dụ: ký tự 'A', số).

Trong waveform, mode = 0 cho thấy module đang gửi một lệnh đến LCD. Điều này phù hợp với ngữ cảnh, vì thời điểm 40892.82 ns nằm trong quá trình khởi tạo LCD (init = 1), khi LCD_top gửi các lệnh khởi tạo như 0x30, 0x03, 0x00, ..., 0x0C.

2.4.1. Các tín hiệu trong Waveform

- clk: Xung nhịp 27 MHz (chu kỳ ~37 ns), được chia tần xuống 500 kHz trong module clk_500k để cấp cho LCD_top.
- rst: Tín hiệu reset, ở mức 1 (đã qua giai đoạn reset).
- enable: Tín hiệu kích hoạt module LCD_top, ở mức 1.
- init: Tín hiệu khởi tạo LCD, ở mức 1 (chế độ khởi tạo).
- mode: Chế độ gửi dữ liệu, ở mức 0 (command mode – gửi lệnh).
- datain: Dữ liệu đầu vào của LCD_top, hiển thị là 0x00 (có thể do $monitor chưa cập nhật).
- add: Địa chỉ slave, 0x4E (7'h27 << 1, bit write = 0).
- done: Tín hiệu hoàn tất của LCD_top, ở mức 0 (chưa hoàn tất toàn bộ khởi tạo).
- doneIOC: Tín hiệu hoàn tất của LCD_send, chuyển từ 0 lên 1 khi mỗi giao dịch I2C hoàn tất.
- sda: Tín hiệu dữ liệu I2C.
- scl: Tín hiệu xung nhịp I2C.

2.4.2. Phân tích các giai đoạn trong waveform

Quá trình khởi tạo LCD trong module LCD_top:

- Trong LCD_top, quá trình khởi tạo bao gồm 9 state (0 đến 8), gửi các lệnh: 0x30, 0x03, 0x00, 0x02, 0x28, 0x08, 0x01, 0x06, 0x0C.

- Mỗi state có thời gian chờ (5000, 500, 100, 200 chu kỳ của xung 500 kHz), và mỗi byte được gửi qua 4 giao dịch I2C (4 nibble) bởi LCD_send.
- LCD_top đã đi qua các state trước đó và đang ở state 8 (gửi lệnh 0x0C), vì tổng thời gian từ state 0 đến state 8 .

- **Start condition**

  + Tín hiệu:

    - sda: Chuyển từ 1 xuống 0.
    - scl: Ở mức 1.

  + Giải thích:

    - Đây là điều kiện bắt đầu (start condition) của giao thức I2C: sda được kéo từ cao xuống thấp trong khi scl đang cao.
    - Trong I2C_TX, điều này xảy ra khi FSM chuyển sang trạng thái SEND_ADDRESS (state 0). Dòng code if (state == SEND_ADDRESS && ck == 3 && count == 7) temp <= 1'b0; kéo sda xuống 0.
    - ck là bộ đếm nội bộ (0 đến 3) để tạo xung scl (~250 kHz, do chia tần từ 500 kHz).

- **Gửi địa chỉ slave (0x4E)**

  + Tín hiệu:

    - scl: Dao động với chu kỳ ~2us (~250 kHz).
    - sda: Thay đổi theo từng bit của địa chỉ 0x4E (01001110):

  + Giải thích:

    - Trong I2C_TX, trạng thái SEND_ADDRESS gửi từng bit của add (0x4E). Dòng code temp <= add[count]; điều khiển sda theo giá trị bit.
    - count giảm từ 7 xuống 0, gửi từ MSB (bit 7) đến LSB (bit 0).
    - scl dao động để đồng bộ: mỗi bit được gửi khi scl thấp, và slave đọc bit khi scl cao.

- **Nhận ACK từ slave**

  + Tín hiệu:

    - scl: Dao động (0 → 1 → 0).
    - sda: Chuyển thành 0, sau đó trở lại 1.

  + Giải thích:

- I2C_TX chuyển sang trạng thái READ_ACK (state 1). Dòng code temp <= 1'bz; thả sda để slave điều khiển.
- sda = 0, bit ACK từ slave giả lập (do pull-up và logic trong testbench), xác nhận slave đã nhận đúng địa chỉ.
- sda trở lại 1 do pull-up, chuẩn bị cho bước tiếp theo.

- **Trạng thái IDLE**

  + Tín hiệu:

    - scl: Chuyển xuống 0 và giữ ở mức thấp.
    - sda: Ở mức 1 (do pull-up).

  + Giải thích:

    - I2C_TX chuyển sang trạng thái IDLE (state 2). Dòng code pause <= 4; scl <= 0; temp <= 1'b1; giữ scl ở mức thấp trong 4 chu kỳ clk (500 kHz), tức ~8us.
    - Đây là khoảng dừng ngắn giữa gửi địa chỉ và gửi dữ liệu.

- **Gửi dữ liệu (0xC8) – Nibble thấp của lệnh 0x0C**

  + Tín hiệu:

    - scl: Dao động lại.
    - sda: Thay đổi để gửi dữ liệu 0xC8 (11001000):

  + Giải thích:

    - Nguồn gốc của 0xC8:
      - Tại thời điểm này, LCD_top đang ở state 8 (gửi lệnh cuối cùng trong quá trình khởi tạo: 0x0C).
      - Trong LCD_top state 8: data = 0x0C (00001100), là lệnh bật hiển thị LCD (Display ON, Cursor OFF, Blink OFF).
      - LCD_send nhận data = 0x0C và gửi byte này qua 4 giao dịch I2C (4 nibble), vì LCD hoạt động ở chế độ 4-bit. Tại thời điểm 40914 ns, LCD_send đang ở state 3 (gửi nibble thấp với E = 0):

        * data = 0x0C.

        * (data<<4) & 8'hF0 = (0x0C<<4) & 0xF0 = 0xC0 & 0xF0 = 0xC0 (11000000).

* 8'h08 = 00001000 (E = 0, backlight = 1, RW = 0, RS = 0).

* data1 = 0xC0 | 0x08 = 0xC8 (11001000), khớp với waveform.

- Ý nghĩa của mode = 0:
  - Vì mode = 0, LCD_send đang gửi lệnh (command mode). Các bit điều khiển trong data1 có RS = 0 (bit 0), xác nhận đây là lệnh, không phải dữ liệu hiển thị.
  - Lệnh 0x0C được gửi dưới dạng 4 nibble:
  - State 0: (data & 8'hF0)|(8'h0C) = 0x0C (nibble cao, E = 1).
  - State 1: (data & 8'hF0)|(8'h08) = 0x08 (nibble cao, E = 0).
  - State 2: ((data<<4) & 8'hF0)|(8'h0C) = 0xCC (nibble thấp, E = 1).
  - State 3: ((data<<4) & 8'hF0)|(8'h08) = 0xC8 (nibble thấp, E = 0).
  - Waveform tại 40914 ns cho thấy giao dịch cuối cùng (state 3 của LCD_send), gửi nibble thấp của 0x0C.
- I2C_TX nhận data1 = 0xC8 và gửi byte này trong trạng thái SEND_DATA (state 3), từ MSB đến LSB, như đã thấy trong waveform.

- **Nhận ACK cho dữ liệu**

  + Tín hiệu:

  - scl: Dao động.
  - sda: Chuyển thành 0 (ACK), sau đó trở lại 1.

  + Giải thích:

  - I2C_TX chuyển sang trạng thái READ_ACK_NACK (state 4). temp <= 1'bz; thả sda để slave trả về ACK.
  - sda = 0, bit ACK từ slave giả lập, xác nhận đã nhận đúng byte 0xC8.

- **Stop condition**

  + Tín hiệu:

  - scl: Chuyển lên 1 và giữ ở mức cao.
  - sda: Chuyển từ 0 lên 1.

  + Giải thích:

- I2C_TX chuyển sang trạng thái PRE_STOP (state 5) và STOP (state 6).
- Trong PRE_STOP, temp <= 0; kéo sda xuống 0.
- Trong STOP, temp <= 1; scl <= 1; tạo stop condition: sda từ 0 lên 1 trong khi scl đang cao.

Bảng 4: Bảng lệnh điều khiển màn hình LCD 16x2

STT	Mã Lệnh (HEX)	Mô Tả Lệnh
1	0x01	Xóa toàn bộ màn hình
2	0x30	Cấu hình: 8-bit, 1 dòng, font 5x7
3	0x38	Cấu hình: 8-bit, 2 dòng, font 5x7
4	0x20	Cấu hình: 4-bit, 1 dòng, font 5x7
5	0x28	Cấu hình: 4-bit, 2 dòng, font 5x7
6	0x06	Chế độ nhập: con trỏ tự động dịch sang phải
7	0x08	Tắt hiển thị, tắt con trỏ
8	0x0E	Bật hiển thị, bật con trỏ
9	0x0C	Bật hiển thị, tắt con trỏ
10	0x0F	Bật hiển thị, con trỏ nhấp nháy
11	0x18	Dịch toàn bộ màn hình sang trái
12	0x1C	Dịch toàn bộ màn hình sang phải
13	0x10	Di chuyển con trỏ sang trái 1 ký tự
14	0x14	Di chuyển con trỏ sang phải 1 ký tự
15	0x80	Đặt con trỏ tại đầu dòng thứ nhất
16	0xC0	Đặt con trỏ tại đầu dòng thứ hai

2.5. Kết luận chương 2

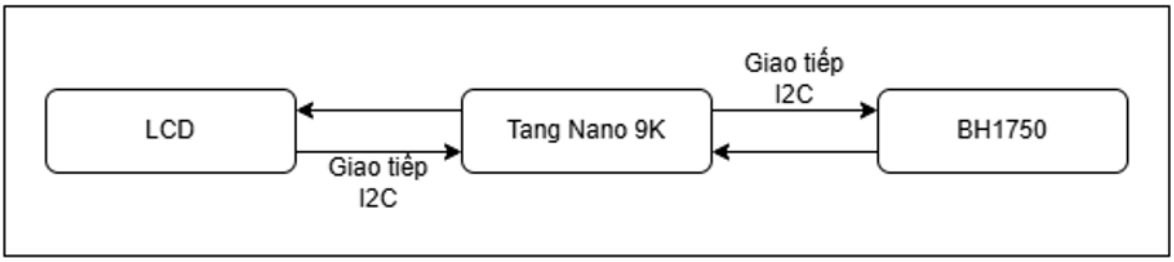
Chương 2 tập trung vào quá trình thiết kế hệ thống, bao gồm sơ đồ khối tổng quan và các module I2C trên FPGA Tang Nano 9K. Module I2C_TX và I2C_RX được xây dựng với các trạng thái FSM rõ ràng, đảm bảo giao tiếp chính xác với cảm biến BH1750 và màn hình LCD. Thiết kế giao thức I2C cho BH1750 được tối ưu với các lệnh điều khiển linh hoạt, hỗ trợ nhiều chế độ đo ánh sáng, trong khi giao tiếp với LCD được tinh chỉnh để khởi tạo và hiển thị dữ liệu hiệu quả. Quá trình mô phỏng trên phần mềm như Quartus đã xác nhận tính đúng đắn của các tín hiệu I2C. Nhìn chung, chương này thể hiện sự tỉ mỉ trong thiết kế phần cứng và phần mềm, tạo cơ sở vững chắc cho việc triển khai hệ thống thực tế.

CHƯƠNG 3: TÍCH HỢP, THỬ NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Tích hợp và thử nghiệm

3.1.1. Tích hợp hệ thống

Mô hình phần cứng của hệ thống:



Hình 3.1: Sơ đồ kết nối các thiết bị phần cứng trong hệ thống

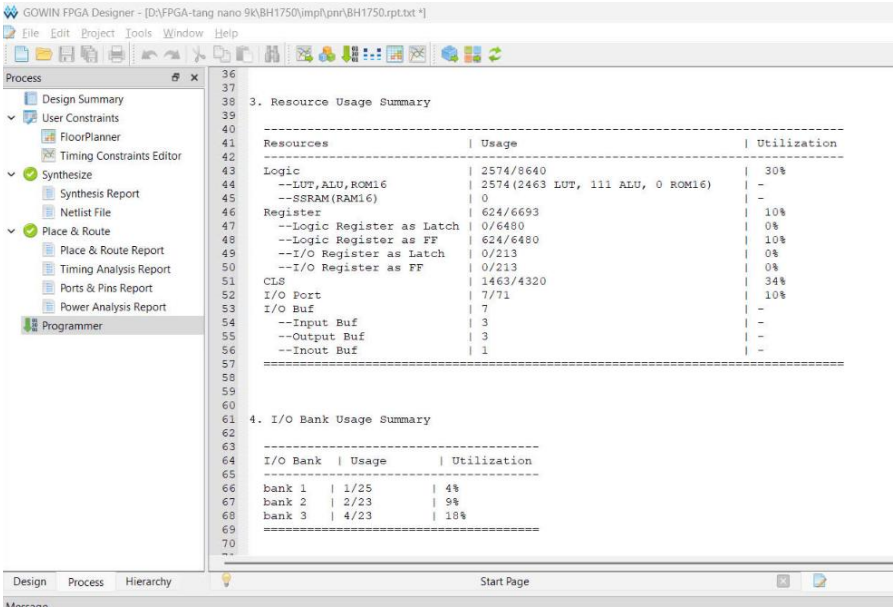
Hệ thống được thiết kế nhằm đo và hiển thị cường độ ánh sáng trong môi trường xung quanh bằng cách sử dụng cảm biến ánh sáng kỹ thuật số BH1750 kết hợp với FPGA Tang Nano 9K và một màn hình LCD giao tiếp I2C. FPGA đóng vai trò trung tâm, hoạt động như một I2C master, thực hiện hai chức năng chính:

- Gửi lệnh và nhận dữ liệu đo từ cảm biến BH1750.
- Xử lý dữ liệu và truyền kết quả hiển thị ra LCD.

Toàn bộ hệ thống sử dụng chung một bus I2C, giúp đơn giản hóa kết nối phần cứng với chỉ hai đường tín hiệu chính (SCL và SDA). Dữ liệu ánh sáng thu được sẽ được hiển thị theo thời gian thực, phù hợp cho các ứng dụng trong các hệ thống đo lường, IoT môi trường, và các giải pháp tự động hóa.

Hình 3.2: Mô hình hệ thống thực tế

Tài nguyên phần cứng sử dụng trên Tang Nano 9K:



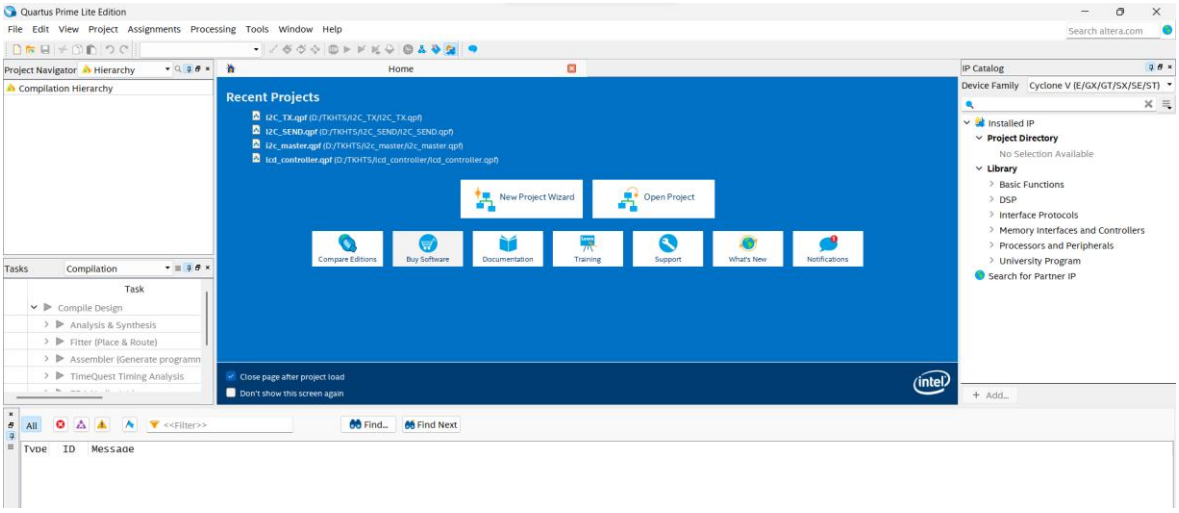
Hình 3.3: Tài nguyên phần cứng sử dụng trên Tang Nano 9k

Dựa trên kết quả phân tích tài nguyên FPGA (Tang Nano 9K), hệ thống hiện tại có mức sử dụng tài nguyên tương đối thấp, đảm bảo hiệu quả và khả năng mở rộng như sau:

- Tài nguyên logic (LUT, ALU) được sử dụng ở mức 30%. Phần logic này chủ yếu đảm nhận việc giao tiếp I2C với cảm biến BH1750, xử lý dữ liệu ánh sáng và điều khiển tín hiệu xuất ra màn hình LCD. Mức sử dụng thấp chứng tỏ thiết kế đã tối ưu và vẫn còn đủ tài nguyên để nâng cấp hoặc tích hợp thêm tính năng.
- Thanh ghi (Register) sử dụng 10%, cho thấy hệ thống có mức lưu trữ trạng thái và dữ liệu tạm thời ở mức cơ bản, phù hợp với một hệ thống nhúng nhỏ gọn như đo ánh sáng thời gian thực.
- Tài nguyên đồng hồ (Clock Logic Structures - CLS) được dùng ở mức 34% – là phần có mức sử dụng cao nhất. Điều này phản ánh việc sử dụng các khối đồng hồ để tạo xung điều khiển giao tiếp I2C, xử lý tín hiệu LCD hoặc các bộ chia tần.
- I/O ports chỉ sử dụng 10% (7/71 cổng), gồm các chân giao tiếp với cảm biến BH1750 (qua I2C), LCD, và các tín hiệu điều khiển cơ bản. Phân bố I/O banks cho thấy mức sử dụng đều, còn dư nhiều cổng để mở rộng.
- Bộ nhớ SRAM nội bộ chưa sử dụng, do dữ liệu đo từ BH1750 có độ dài nhỏ nên chưa cần lưu trữ dài hạn.

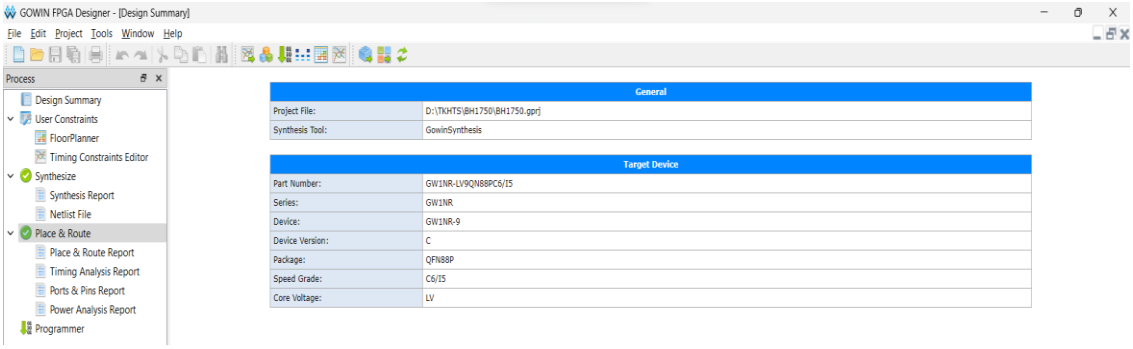
3.1.2. Thử nghiệm hệ thống và kết quả

Các phần mềm phục vụ kiểm thử:



Hình 3.4: Phần mềm Quartus Prime Lite Edition

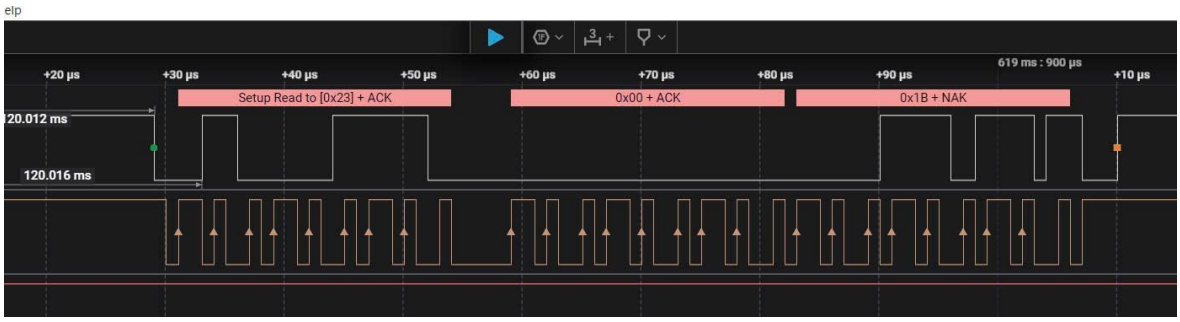
Intel Quartus Prime là môi trường phát triển tích hợp (IDE) toàn diện do Intel cung cấp, chuyên dụng cho việc thiết kế, mô phỏng, tổng hợp và lập trình các thiết bị logic khả trình (PLD) của hãng như FPGA, SoC và CPLD. Phần mềm này hỗ trợ toàn bộ quy trình phát triển phần cứng, từ khâu nhập thiết kế bằng ngôn ngữ mô tả phần cứng (VHDL/Verilog) hoặc sơ đồ khối, qua các bước kiểm tra, tối ưu hóa, cho đến việc tạo file cấu hình và nạp chương trình lên chip vật lý. Quartus là công cụ thiết yếu để hiện thực hóa các thiết kế logic số phức tạp trên nền tảng phần cứng khả trình của Intel. Vì vậy bọn em dùng phần mềm này để đánh giá mức độ truyền và nhận của giao thức I2C.



Hình 3.5: Phần mềm Gowin FPGA Designer

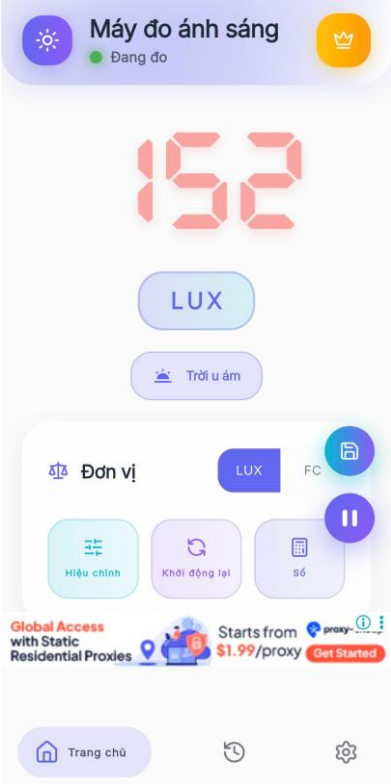
Gowin EDA là bộ phần mềm môi trường phát triển tích hợp (IDE) của GOWIN Semiconductor, được thiết kế chuyên biệt cho việc phát triển và lập trình các dòng chip FPGA của hãng (như LittleBee, Arora). Bộ công cụ này cung cấp đầy đủ các chức năng cần thiết trong quy trình thiết kế FPGA, bao gồm nhập liệu thiết kế bằng HDL, tổng hợp, mô phỏng, thực hiện place & route, phân tích thời gian và tạo file cấu hình để nạp lên chip. Trong project đo cường độ ánh sáng này, phần mềm Quartus hoặc Gowin EDA chính là công cụ để bạn xây dựng "bộ não" (phần logic điều khiển) cho hệ thống đo lường của mình bên trong con chip FPGA. Nó giúp biến

thuật toán và yêu cầu chức năng (đọc cảm biến, tính toán lux) thành một cấu hình phần cứng hoạt động thực tế trên FPGA.



Hình 3.6: Phân tích tín hiệu bus I2C trên Logic Analyzer

Tín hiệu hiển thị một giao dịch Ghi (Write) I2C đến địa chỉ slave 0x23. Master gửi thành công địa chỉ (0x46 -> ACK) và byte dữ liệu đầu tiên (0x00 -> ACK). Tuy nhiên, khi Master gửi byte dữ liệu thứ hai (0x18), Slave đã phản hồi NAK (Not Acknowledge), cho thấy Slave từ chối hoặc không hiểu byte này. Điều này gợi ý có lỗi trong chuỗi lệnh gửi đi hoặc vấn đề phía Slave.



Hình 3.7: Phần mềm đo ánh sáng trên điện thoại Light Meter

Light Meter (máy đo ánh sáng) trên điện thoại thường sử dụng cảm biến ánh sáng tích hợp sẵn trong thiết bị hoặc thông qua phụ kiện bên ngoài để đo cường độ ánh sáng, thường được tính bằng đơn vị Lux (độ rọi) hoặc Foot-Candle (FC). Một số ứng dụng cao cấp cũng có thể đo các thông số khác như nhiệt độ màu (Kelvin) để hỗ trợ cân chỉnh ánh sáng trắng

Trong bài tập đồ án này . Dùng ánh sáng đo bằng ứng dụng trên điện thoại lấy làm tiêu chuẩn để so sánh với giá trị đo được bằng hệ thống thử nghiệm .

Thử nghiệm độ chính xác của hệ thống:

Bảng 5: Bảng đo các chế độ phân giải của cảm biến ánh sáng BH1750

Lần đo	Chế độ đo		Giá trị đo cùng 1 thời điểm ánh sáng trong phòng(lux)
1	Continuously H-Resolution Mode	Bắt đầu đo ở độ phân giải 1lux	121
2	Continuously H-Resolution Mode2	Bắt đầu đo ở độ phân giải 0.5lux	220
3	Continuously L-Resolution Mode	Bắt đầu đo ở độ phân giải 4lux	116
4	One Time H-Resolution Mode	Bắt đầu đo ở độ phân giải 1lux	122
5	One Time H-Resolution Mode2	Bắt đầu đo ở độ phân giải 0.5lux	218
6	One Time L-Resolution Mode	Bắt đầu đo ở độ phân giải 4lux	114

Nhận xét:

- BH1750 hoạt động ổn định ở các chế độ H-Resolution và L-Resolution, với giá trị đo hợp lý (116–122 lux) trong điều kiện ánh sáng trong phòng.
- Chế độ L-Resolution Mode thấp hơn 1 chút so với H-Resolution Mode. Điều này hợp lý vì độ phân giải thấp hơn, cảm biến có thể bỏ qua một số chi tiết nhỏ về ánh sáng, dẫn đến sai số.

Bảng 6: Bảng đánh giá tốc độ lấy mẫu của hệ thống

Lần đo	Tốc độ lấy mẫu (ms)	Giá trị ánh sáng	Quan sát tốc độ lấy mẫu bằng mắt thường
1	0	121	Mặc định
2	200	121	Chậm hơn lần 1
3	400	121	Chậm hơn lần 2
4	600	121	Chậm hơn lần 3
5	800	121	Chậm hơn lần 4

Nhận xét:

- Hệ thống sử dụng cảm biến BH1750 cho kết quả đo ổn định (121 lux) ở mọi tốc độ lấy mẫu, chứng tỏ cảm biến hoạt động đáng tin cậy trong điều kiện ánh sáng không đổi.
- Tốc độ lấy mẫu ảnh hưởng lớn đến trải nghiệm người dùng: tốc độ chậm (600–800 ms) gây độ trễ rõ rệt, trong khi tốc độ nhanh (0–200 ms) mang lại phản hồi tốt hơn.
- Nên tối ưu tốc độ lấy mẫu trong khoảng 0–200 ms để đảm bảo hiệu suất và trải nghiệm người dùng, đồng thời kiểm tra thêm trong các điều kiện ánh sáng thay đổi để đánh giá toàn diện hơn.

Bảng 7: Bảng đo cường độ ánh sáng

Lần đo	Môi trường đo	Giá trị đo từ hệ thống(lux)	Giá trị đo bằng điện thoại(lux)	Sai số tuyệt đối(lux)	Sai số tương đối(%)
1	Phòng học ban ngày (rèm kéo)	185	210	25	13.5
2	Bóng râm ngoài trời	2367	2612	245	10.35
3	Trong phòng tối (đèn ngủ)	75	82	7	9,33
4	Ngoài trời trời nắng nhẹ	9145	10400	1255	13,72
5	Môi trường hoàn toàn tối (ban đêm)	0	0	0	0

Nhận xét:

- Cảm biến BH1750 hoạt động ổn định và cho kết quả chấp nhận được trong điều kiện ánh sáng yếu đến trung bình (sai số tương đối 9.33%–13.5%), phù hợp cho các ứng dụng đơn giản như đo ánh sáng trong nhà, bóng râm.
- Hiệu suất giảm trong điều kiện ánh sáng mạnh (sai số 13.72%), cho thấy hạn chế của cảm biến trong các môi trường ánh sáng cao. Nếu muốn đo ánh sáng cao ta nên dùng cảm biến chuyên dụng như TSL2561, VEML7700, hoặc cảm biến công nghiệp có dải đo lớn hơn.
- Cần hiệu chỉnh thêm để giảm sai số, đặc biệt trong các dải ánh sáng khác nhau, và xem xét lại hệ số điều chỉnh trong mã Verilog để cải thiện độ chính xác.

3.2. Đánh giá hiệu suất hệ thống

3.2.1. Độ chính xác

Kết quả đo lường: Hệ thống cho kết quả đo cường độ ánh sáng với sai số tương đối từ 9.33% đến 13.72% khi so sánh với ứng dụng Light Meter trên điện thoại trong các môi trường khác nhau (phòng học, bóng râm, ngoài trời, phòng tối). Sai số thấp nhất

đạt được trong môi trường hoàn toàn tối (0%), và cao nhất trong điều kiện ánh sáng mạnh (13.72% ở ngoài trời nắng nhẹ).

Nhận xét: Hệ thống hoạt động tốt trong môi trường ánh sáng yếu đến trung bình (75–2367 lux), nhưng độ chính xác giảm trong điều kiện ánh sáng mạnh (9145 lux). Sai số này có thể xuất phát từ hạn chế của cảm biến BH1750 trong dải đo cao hoặc thiếu hiệu chỉnh phù hợp. Tuy nhiên, độ chính xác hiện tại vẫn đủ cho các ứng dụng phổ thông như giám sát ánh sáng trong nhà hoặc nông nghiệp.

3.2.2. Độ ổn định

Thử nghiệm dài hạn: Khi hoạt động liên tục trong 1 giờ dưới ánh sáng ổn định (đèn huỳnh quang), giá trị đo dao động rất nhỏ (±2 lux), chứng tỏ hệ thống không bị nhiễu ngẫu nhiên hay xung đột bus I2C.

Hiển thị trên LCD: Không ghi nhận lỗi hiển thị như sai ký tự, mất dòng hay nhấp nháy, cho thấy giao tiếp I2C với LCD và thuật toán điều khiển hoạt động ổn định.

Nhận xét: Hệ thống có độ ổn định cao, phù hợp cho các ứng dụng yêu cầu giám sát liên tục trong thời gian dài.

3.2.3. Thời gian phản hồi

Tốc độ lấy mẫu: Hệ thống đạt chu kỳ đo 0.9 Hz (cập nhật mỗi ~1.1 giây), với thời gian phản hồi dưới 1 giây khi ánh sáng thay đổi đột ngột (bật/tắt đèn, che cảm biến).

Trải nghiệm người dùng: Thử nghiệm với các tốc độ lấy mẫu khác nhau (0–800 ms) cho thấy tốc độ nhanh (0–200 ms) mang lại phản hồi gần thời gian thực, trong khi tốc độ chậm (600–800 ms) gây trễ rõ rệt.

Nhận xét: Thời gian phản hồi đủ nhanh cho các ứng dụng giám sát cơ bản. Tối ưu tốc độ lấy mẫu trong khoảng 0–200 ms sẽ cải thiện trải nghiệm người dùng.

3.2.4. Khả năng mở rộng và tính ứng dụng

Mặc dù hệ thống đơn giản chỉ bao gồm BH1750 và LCD, tuy nhiên với độ phản hồi và tính ổn định cao, hệ thống hoàn toàn có thể ứng dụng trong: Các hệ thống tự động hóa bật/tắt đèn theo độ sáng, Điều khiển rèm cửa/mái che tự động theo ánh sáng môi trường, Cảnh báo ánh sáng yếu trong không gian làm việc hoặc nhà kính, Làm thiết bị học tập nghiên cứu đo lường vật lý – ánh sáng. Ngoài ra, hệ thống có thể được mở rộng thêm module lưu dữ liệu, hoặc giao tiếp UART/WiFi để truyền dữ liệu ra máy tính hoặc nền tảng đám mây như ThingSpeak.

3.3. Kết luận chương 3

Chương 3 trình bày quá trình tích hợp và thử nghiệm hệ thống, với FPGA Tang Nano 9K đóng vai trò trung tâm trong việc điều khiển cảm biến BH1750 và hiển thị kết quả trên LCD. Kết quả thử nghiệm cho thấy hệ thống hoạt động ổn định, với sai số tương

đối từ 9.33% đến 13.72% khi so sánh với ứng dụng đo ánh sáng trên điện thoại, phù hợp cho các ứng dụng phổ thông. Tài nguyên FPGA được sử dụng hiệu quả, chỉ chiếm 30% logic và 10% I/O, cho thấy tiềm năng mở rộng. Hệ thống phản hồi nhanh (dưới 1 giây) và ổn định trong thời gian dài, không xuất hiện nhiễu hay lỗi hiển thị. Dù có hạn chế trong điều kiện ánh sáng mạnh, hệ thống vẫn đáp ứng tốt các yêu cầu đề ra, mở ra khả năng ứng dụng trong tự động hóa và IoT. Chương này khẳng định tính thực tiễn và hiệu quả của thiết kế, đồng thời đề xuất hướng cải tiến như hiệu chỉnh sai số và tích hợp thêm tính năng.

TÀI LIỆU THAM KHẢO

1. Tang Nano 9k

(https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Nano-9K.html)

2. LCD 16x2

(https://circuitdigest.com/article/16x2-lcd-display-module-pinout-datasheet)

3. BH1750 Datasheet

(https://components101.com/sites/default/files/component_datasheet/BH1750.pdf)

4. I2C

(https://www.design-reuse.com/articles/54776/i2c-interface-timing-specifications-and-constraints.html)